MongoDB

Ecosistema MongoDB

Giorno 2 · mattina

Marco De Giovanni · 21 ottobre 2026

01

CICLO 1 · TEORIA

Paradigma, BSON, insert, find

Agenda e obiettivi

Mattina

Teoria 1	paradigma, BSON, insert, find
Lab 1	caricamento e interrogazione
Teoria 2	update, validazione, architettura
Lab 2	modifica, validazione, replica

Pomeriggio

Teoria 3	modellazione, transazioni
Lab 3	modellazione
Teoria 4	indici, `explain()`, aggregation, confronto con PostgreSQL
Lab 4	indici e aggregation

A FINE GIORNATA SAPRETE

01 interrogare e modificare documenti

02 scegliere tra embedding e referencing

03 leggere un `explain()` e scegliere l'indice

04 scrivere pipeline equivalenti alle query SQL del Giorno 1

05 decidere tra PostgreSQL e MongoDB

02 CICLO 1 · INTRODUZIONE

Ambiente, Compass e dataset

- Connessione: `mongodb://localhost:27018/?directConnection=true`. Prima istruzione di ogni sessione: `use shop`.
- Dataset `shop` (seed fisso, **uguale su tutte le macchine**):

COLLEZIONE	DOCUMENTI	CORRISPONDENTE NEL GIORNO 1
`customers`	100.000	`customers`
`orders`	1.000.000	`orders` + `order_items` (le righe sono **dentro** l'ordine)

MONGOSH

```
{ _id: 1, customerId: 44830, customer: { name: 'Customer 44830', country: 'IT' },
  status: 'delivered', total: Decimal128('866.82'),
  createdAt: ISODate('2026-01-01T00:00:30.000Z'), tags: [], attrs: { channel: 'app' },
  items: [ { lineNo: 1, sku: 'SKU-171', qty: 4, unitPrice: Decimal128('18.48') },
           { lineNo: 2, sku: 'SKU-106', qty: 2, unitPrice: Decimal128('63.47') } ] }
```

N1 CICLO 1 · PARADIGMA NOSQL

Perché NoSQL e le quattro famiglie

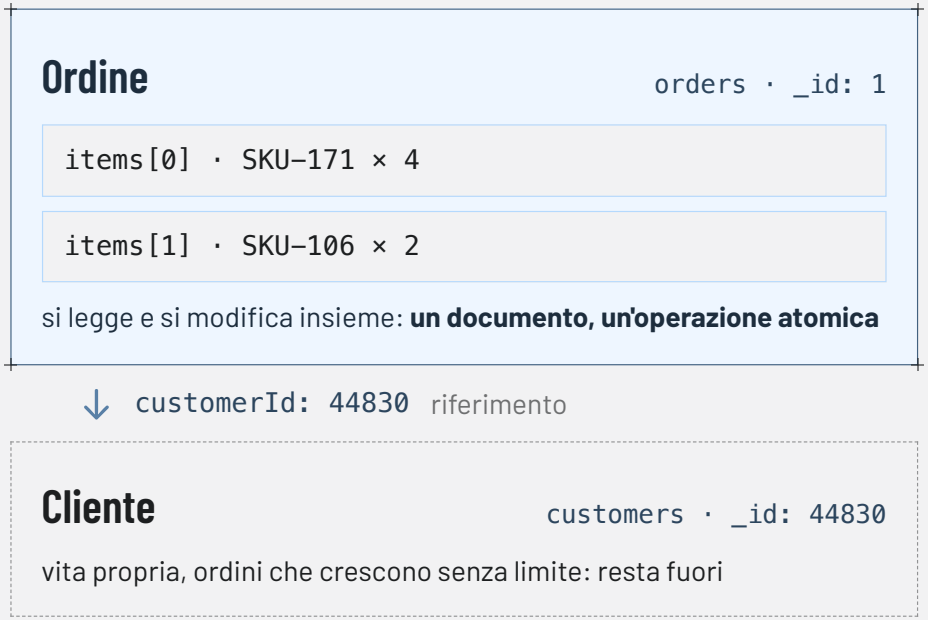
+ **NoSQL = "Not only SQL"**. Le spinte: l'**impedance mismatch** (un oggetto applicativo scomposto in più tabelle e ricomposto con i `JOIN`), lo schema rigido, la scalabilità orizzontale, i dati eterogenei.
+ Oggi i due mondi convergono: PostgreSQL ha `jsonb`, MongoDB ha transazioni e validazione dello schema.

FAMIGLIA	MODELLO	ESEMPI	USO TIPICO
Key-value	chiave → valore opaco	Redis, DynamoDB	cache, sessioni
Documentale	chiave → documento interrogabile	**MongoDB**	aggregati applicativi, cataloghi
Colonnare	righe con famiglie di colonne, partizionate	Cassandra	scritture massive, serie temporali
Grafo	nodi e archi	Neo4j	reti, raccomandazioni

Il modello documentale e l'aggregato

- **Documento**: albero di campi, sottodocumenti e array, salvato in **BSON**.
- **Aggregato**: insieme di oggetti che si leggono, si modificano e restano consistenti **insieme**. Un ordine con le sue righe lo è. Un cliente con tutti i suoi ordini no.
- In MongoDB un'operazione su **un solo documento è atomica**. Se il documento coincide con l'aggregato, l'atomicità è gratuita.

AGGREGATO E RIFERIMENTO



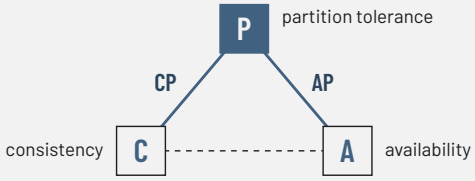
Quando conviene il documentale, e quando no

☑ CONVIENE	☒ NON CONVIENE
dati letti e scritti per aggregato	query ad hoc su molte entità (reporting)
schema variabile o in evoluzione	invarianti forti tra entità diverse (saldi, giacenze condivise)
pattern di accesso noti in fase di progetto	pattern di accesso ignoti o mutevoli
scritture da scalare in orizzontale	volume che sta su un nodo, modello molto relazionale
payload esterni da conservare così come arrivano	vincoli referenziali garantiti dal database

CAP e PACELC

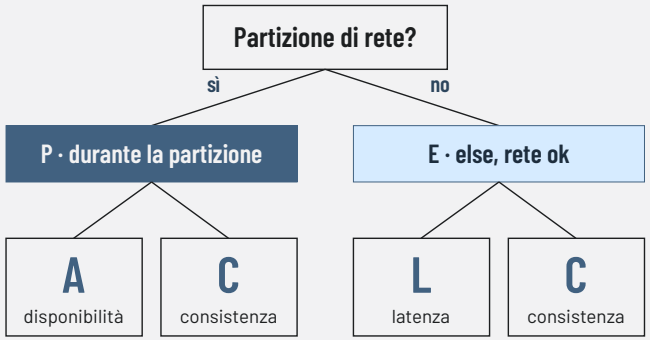
- **CAP**: in un sistema distribuito, durante una **partizione di rete** bisogna scegliere tra **C**onsistency (ogni lettura vede l'ultima scrittura confermata) e **A**vailability (ogni richiesta riceve risposta). La P non è una scelta: le partizioni capitano.
- **Limiti**: CAP descrive solo il comportamento *durante* la partizione, e classifica in modo binario (CP/AP). Un nodo singolo non è un sistema distribuito: CAP non si applica.
- **PACELC**: **P**artition → **A** o **C**; **E**lse (funzionamento normale) → **L**atency o **C**onsistency.
- MongoDB non è "CP" o "AP": lo sono le singole operazioni, in base a **write concern**, **read concern** e **read preference** (si vedono in A6–A7).

CAP

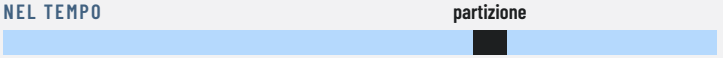


CA: solo senza rete (nodo singolo)

PACELC



NEL TEMPO



quasi sempre funzionamento normale: la scelta che conta è E → L o C

N5 CICLO 1 · PARADIGMA NOSQL

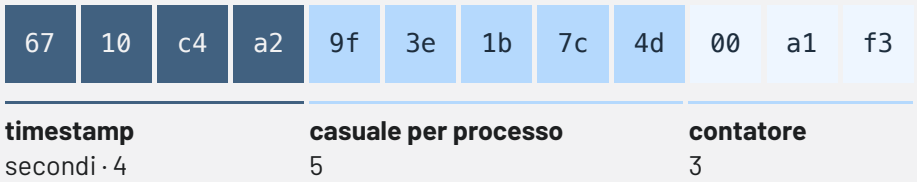
Terminologia SQL ↔ MongoDB

SQL (GIORNO 1)	MONGODB
tabella / riga / colonna	**collezione** / **documento** / **campo**
`PRIMARY KEY`	`_id` (sempre presente, sempre univoco)
`FOREIGN KEY`	nessun equivalente: riferimento gestito dall'applicazione
`JOIN`	embedding oppure `$lookup`
`SELECT … WHERE`	`find(filtro, proiezione)`
`GROUP BY`	`$group` (aggregation)
`CREATE TABLE`	facoltativo: la collezione nasce al primo insert
`CHECK`, `NOT NULL`	`$jsonSchema` (facoltativo)
`EXPLAIN`	`explain()`
view	view (`db.createView()`): pipeline salvata, in sola lettura, ricalcolata a ogni lettura

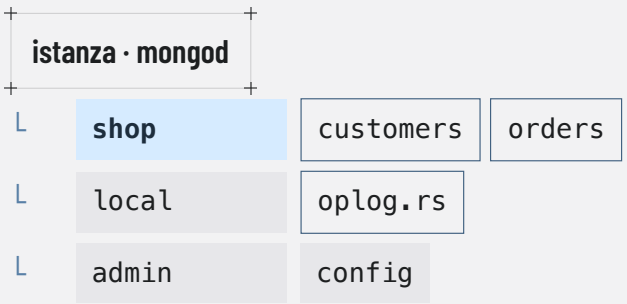
Database, collezioni, documenti, `_id` e ObjectId

- **Istanza** → **database** → **collezioni** → **documenti**. Database di sistema: `admin`, `config`, `local` (che contiene l'**oplog**).
- La collezione **nasce al primo insert**, senza definizione. Namespace: `database.collezione`.
- `_id`: presente in **ogni** documento, univoco, **immutabile**, di qualunque tipo tranne array. Se manca lo genera il driver come **`ObjectId`**.
- `ObjectId`: 12 byte = 4 di timestamp (secondi) + 5 casuali per processo + 3 di contatore. Generato dal **client**, approssimativamente crescente nel tempo.

OBJECTID · 12 BYTE



GERARCHIA



Il linguaggio di query: MQL, non JavaScript

01 MongoDB non ha un linguaggio testuale come SQL: le query sono **documenti** (MQL, MongoDB Query Language). Il server riceve **BSON**, non codice.

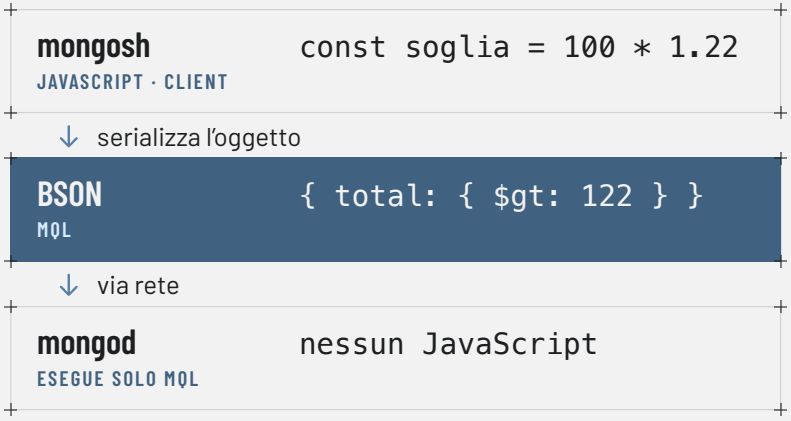
02 **mongosh** è una shell **JavaScript**: il JS costruisce l'oggetto filtro ed è eseguito **nel client**. Il server esegue solo il documento.

03 Lo stesso filtro, dal driver Python o Java, è un dizionario o un `Document`: la sintassi cambia, la query no.

04 Non confondere: `$expr` e la pipeline di aggregazione sono **operatori MQL**, non JS. `$where` (JS lato server) è deprecato e lento.

MONGOSH

```
const soglia = 100 * 1.22
db.orders.find({ total: { $gt: soglia } })
db.orders.find({
  $expr: { $gt: ['$total', '$budget'] }
})
```



Tipi BSON e la trappola dei numeri

- BSON = JSON binario con più tipi: `double`, `int`, `long`, `decimal`, `string`, `bool`, `date`, `objectId`, `null`, `object`, `array`, `binData`.
- In `mongosh` il tipo numerico **dipende dal valore**: intero a 32 bit → `int`; con decimali o fuori intervallo → `double`. Lo stesso campo può cambiare tipo da documento a documento.
- La stampa non mostra la differenza: serve `$type`.

MONGOSH

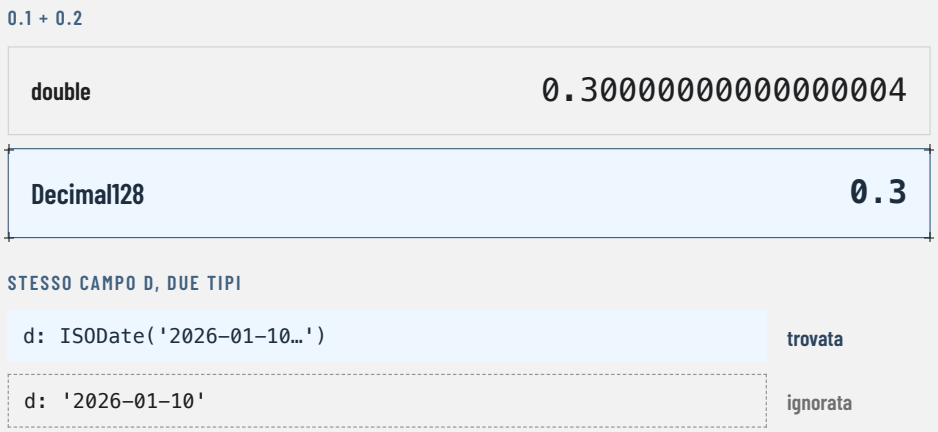
```
db.lab_types.insertMany([ { _id: 'int', v: 1 }, { _id: 'double', v: 1.5 }, { _id: 'big', v: 2147483648 },
                          { _id: 'long', v: NumberLong('1') }, { _id: 'string', v: '1' } ])
db.lab_types.aggregate([ { $project: { t: { $type: '$v' } } } ])
```

OUTPUT (STATICO)

```
[ { _id: 'int', t: 'int' }, { _id: 'double', t: 'double' }, { _id: 'big', t: 'double' },
  { _id: 'long', t: 'long' }, { _id: 'string', t: 'string' } ]
```

Importi e date: Decimal128 e Date

- `double` = virgola mobile binaria: `0.1 + 0.2 ≠ 0.3`. Per gli **importi**: `Decimal128` (34 cifre decimali esatte), sempre con la **stringa**: `NumberDecimal('19.90')`. In Java: `BigDecimal` → `Decimal128`.
- `Date`: millisecondi dall'epoch, **sempre UTC**. Una data scritta come stringa **non** si confronta con una `Date`.



MONGOSH

```
// 0.30000000000000004
0.1 + 0.2
db.lab_types.aggregate([ { $match: { _id: 'money' } }, { $project: { sum: { $add: ['$a', '$b'] } } } ])
// con a = NumberDecimal('0.1') e b = NumberDecimal('0.2') → sum: Decimal128('0.3')
// trova la Date, non la stringa '2026-01-10'
db.lab_types.find({ d: { $gt: ISODate('2026-01-01T00:00:00Z') } })
// 'string': senza new, Date() restituisce una stringa
typeof Date()
```

B4 CICLO 1 · BSON E INSERT

Documenti annidati, array, dot notation, 16 MB

- Un campo può contenere un **sottodocumento** (`attrs`) o un **array** (`tags`, `items`). Si raggiunge con la **dot notation**: `'attrs.channel'`, `'items.sku'`, `'items.0.sku'`.
- Il confronto con un sottodocumento **intero** (`{ attrs: { channel: 'web' } }`) esige lo stesso contenuto **nello stesso ordine**: non si usa quasi mai.
- Limite di **16 MB** per documento; `bsonsize(doc)` restituisce i byte (`mongosh`). I nomi dei campi si ripetono in ogni documento.

MONGOSH

```
// 333333
db.orders.countDocuments({ 'attrs.channel': 'web' })
// 300000: esclude gli ordini web con coupon o risk
db.orders.countDocuments({ attrs: { channel: 'web' } })
// 348
bsonsize(db.orders.findOne({ _id: 1 }))
```

Insert: creazione implicita, `insertMany`, `ordered`

- `insertOne`, `insertMany`. La collezione nasce al primo insert: **un errore di battitura nel nome crea una nuova collezione**, senza errori (`lab_custmers`).
- Ogni insert su un documento è atomico, **il batch no**. `ordered` (default `true`) decide cosa succede dopo un errore:
- L'errore (`MongoBulkWriteError`, codice **11000** per le chiavi duplicate) contiene `insertedCount` e `writeErrors` con l'**indice** del documento nel batch.

ESEMPIO · _ID: 1 RIPETUTO

```
db.lab_customers.insertMany([
  { _id: 1, name: 'Anna' },
  { _id: 1, name: 'Bruno' },   // duplicato
  { _id: 3, name: 'Carla' }
], { ordered: false })

// MongoBulkWriteError
// insertedCount: 2
// writeErrors: [{ index: 1, code: 11000 }]
```



F1 CICLO 1 · FIND

Find: filtri, operatori e la regola dei negativi

- `find(filtro, proiezione)` restituisce un **cursore** (la shell mostra 20 documenti, `it` per continuare). Senza `sort` l'ordine **non è garantito**.
- Confronto: `$eq $ne $gt $gte $lt $lte $in $nin`. Logici: più condizioni nello stesso filtro = **AND implicito**; `$or`, `$nor`, `$not`, `$and` esplicito.
- **Regola**: gli operatori **negativi** (`$ne`, `$nin`, `$nor`, `$not`) **includono i documenti senza il campo**.
- Trappola JavaScript: una chiave ripetuta nello stesso oggetto **si sovrascrive**.

MONGOSH

```
// è solo { tags: 'promo' }
{ tags: 'gift', tags: 'promo' }
// 999800: includono gli ordini senza risk
db.orders.countDocuments({ 'attrs.risk': { $nin: ['high'] } })
// 0
db.orders.countDocuments({ 'attrs.risk': { $exists: true, $ne: 'high' } })
```

F1B CICLO 1 · FIND

Carrellata di filtri e un esempio combinato

```
db.orders.find({
  status: { $in: ['paid', 'shipped'] },                                  // STATO  ·  status IN ('paid', 'shipped')
  total: { $gte: NumberDecimal('500') },                                 // IMPORTO  ·  total >= 500
  createdAt: { $gte: ISODate('2026-03-01'), $lt: ISODate('2026-04-01') }, // INTERVALLO  ·  created_at in [mar, apr)
  'customer.country': 'IT',                                              // ANNIDATO  ·  JOIN customers … country = 'IT'
  $or: [ { 'attrs.channel': 'app' }, { tags: 'gift' } ]                  // ALTERNATIVA  ·  channel = 'app' OR 'gift' IN tags
})
```

+ Esempio combinato: le condizioni si sommano nello stesso documento (AND implicito); `$or` è un campo come gli altri.

F2 CICLO 1 · FIND

Campo assente e `null`; `$exists` e `$type`

FILTRO	VALORE `'WELCOME'`	NULL `null`	ASSENTE —	ASSENTE —	TROVA
`{ coupon: null }`	☐	■	■	■	campo `null` **oppure assente**
`{ coupon: { $exists: false } }`	☐	☐	■	■	solo campo assente
`{ coupon: { $type: 'null' } }`	☐	■	☐	☐	solo campo che vale `null`
`{ coupon: { $exists: true } }`	■	■	☐	☐	campo presente, **anche se `null`**

+ `$type` filtra per tipo BSON (alias `'number'` = qualunque tipo numerico): serve a trovare i **tipi inconsistenti**.

F3 CICLO 1 · FIND

Array e array di documenti: `$elemMatch`

FILTRO	SIGNIFICATO
`{ tags: 'gift' }`	**almeno un** elemento vale `gift`
`{ tags: ['gift', 'express'] }`	array esatto: stessi elementi, **stesso ordine**
`{ tags: { $all: ['express', 'gift'] } }`	contiene tutti, in qualunque ordine
`{ tags: { $size: 0 } }`	esattamente N elementi (niente intervalli)
`{ 'items.sku': 'A' }`	almeno una riga di `items` ha `sku` = `A`

ORDINE 1 · DUE CONDIZIONI, RIGHE DIVERSE

`items[0] · sku: 'A', qty: 1`	`sku: 'A' ✓`
`items[1] · sku: 'B', qty: 5`	`qty ≥ 5 ✓`

■ Dot notation: l'ordine 1 **passa per errore**

□ `$elemMatch`: entrambe sulla stessa riga, l'ordine 1 è escluso

+ **Trappola**: con due condizioni su un array di documenti, ciascuna può essere soddisfatta da una riga **diversa**. Per la **stessa riga**: `$elemMatch`.

MONGOSH

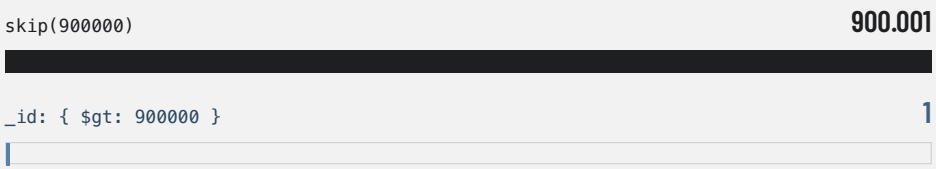
```
// ordini con una riga di A in quantità >= 5: l'ordine 1 ha A×1 e B×5
// → 1 e 2
db.lab_orders.find({ 'items.sku': 'A', 'items.qty': { $gte: 5 } }, { _id: 1 })
// → solo 2
db.lab_orders.find({ items: { $elemMatch: { sku: 'A', qty: { $gte: 5 } } } }, { _id: 1 })
```

F4 CICLO 1 · FIND

Proiezione, ordinamento, paginazione

- Proiezione **inclusiva** (`{ customer: 1, status: 1 }`: solo questi più `_id`) o **esclusiva** (`{ items: 0 }`). Non si mescolano, salvo `_id: 0`.
- `sort({ campo: 1 | -1 })`, `limit(n)`, `skip(n)`. Ordine di applicazione: `sort` → `skip` → `limit`.
- `countDocuments(filtro)` conta eseguendo la query; `estimatedDocumentCount()` legge i metadati.
- **`skip` costa**: per saltare n documenti il server li attraversa. **Paginazione per range**: si filtra dopo l'ultimo valore visto.

CHIAVI ESAMINATE · PAGINA DOPO 900.000



Per range: si riparte dall'ultimo valore visto, senza attraversare i precedenti.

PAGINA "DOPO 900.000"	totalKeysExamined
`sort({ _id: 1 }).skip(900000).limit(1)`	900.001
`find({ _id: { $gt: 900000 } }).sort({ _id: 1 }).limit(1)`	1

MONGOSH

```
// inclusiva: solo nome e totale
db.orders.find({ status: 'delivered' }, { _id: 0, 'customer.name': 1, total: 1 })
// esclusiva: tutto tranne righe e attrs
db.orders.find({}, { items: 0, attrs: 0 })
// i 5 ordini più cari; _id rende l'ordine univoco
db.orders.find().sort({ total: -1, _id: 1 }).limit(5)
// pagina successiva, dopo l'ultimo _id visto
db.orders.find({ _id: { $gt: lastId } }).sort({ _id: 1 }).limit(20)
```

02

CICLO 1 · LABORATORIO

Caricamento e interrogazione

Otto esercizi su ShopIT: inserimento, filtri, ordinamento e paginazione. In chiusura, correzione in plenaria.

L1 CICLO 1 · CONSEGNA

Consegna Laboratorio 1

+ **Fase 1: caricamento e interrogazione** sul database `shop` di ShopIT.

TEMA	#	ESERCIZIO
Inserimento	1	Crea `products` con tre prodotti: `price` in `Decimal128`, `stock` intero, `createdAt` come `Date`. Verifica i tipi con `$type`.
Inserimento	2	Inserisci `{ sku: 'SKU-900', weight: 2 }` e `{ sku: 'SKU-901', weight: 2.5 }`: che tipo ha `weight` in ciascuno? Quanti ne trova `{ weight: { $type: 'double' } }`?
Inserimento	3	`insertMany` di tre prodotti con `_id` duplicato in 2ª posizione, con `ordered: true` e poi `false`: quanti documenti restano? Leggi `writeErrors`.
Filtri	4	Ordini `delivered` dall'Italia con `total` oltre 500 €.
Filtri	5	Ordini che **non** hanno `attrs.risk: 'high'`: confronta `$ne` con `$exists` + `$ne` e spiega la differenza.
Filtri	6	Ordini con **una stessa riga** `SKU-171` e `qty` ≥ 3.
Ordinamento	7	I 10 ordini più recenti del cliente 44830: solo `_id`, `total`, `createdAt`.
Paginazione	8	Pagina 3 (10 per pagina) degli ordini `delivered` per `_id`: con `skip` e per range. Confronta `totalKeysExamined` in `explain`.

03

CICLO 2 · TEORIA

Update, validazione, architettura

Operatori di update e atomicità del documento

- `updateOne(filtro, update)`, `updateMany`. L'update usa **operatori**: `$set`, `$unset`, `$inc`, `$mul`, `$min`, `$max`, `$currentDate`. Un update senza operatori è **rifiutato**; per sostituire un documento intero c'è `replaceOne`.
- Esito: `matchedCount` (trovati) e `modifiedCount` (cambiati davvero: se il valore è già quello, nessuna scrittura).
- Un'operazione su **un documento** è atomica. L'**update condizionale** rende atomici controllo e modifica:

UPDATE CONDIZIONALE · UN'OPERAZIONE

`stock ≥ 2 → $inc: { stock: −2 }`

controllo e modifica insieme, sul server

LEGGI → CONTROLLA → SCRIVI · TRE PASSI

`find → 2` → `if (≥ 2)` → `set: 0`

finestra di concorrenza: un altro client scrive qui

MONGOSH

```
// toglie 2 pezzi solo se ce ne sono almeno 2: un'unica operazione
db.lab_products.updateOne({ _id: 'P-001', stock: { $gte: 2 } }, { $inc: { stock: -2 } })
// se non c'è abbastanza stock: matchedCount: 0, nessuna modifica
```

Operatori sugli array

OPERATORE	EFFETTO
`$push` (+ `$each`, `$position`, `$slice`, `$sort`)	aggiunge uno o più elementi
`$addToSet`	aggiunge **solo se assente** (idempotente)
`$pull`, `$pullAll`, `$pop`	rimuove per valore, per condizione, primo o ultimo
`'items.$.qty'`	**primo** elemento che soddisfa la condizione del filtro
`'items.$[r].flag'` + `arrayFilters`	**tutti** gli elementi che soddisfano la condizione `r`

MONGOSH

```
// ultime 3 recensioni, la più recente in testa (array limitato)
db.lab_products.updateOne({ _id: 'P-001' },
  { $push: { lastReviews: { $each: [ { stars: 5, createdAt: new Date() } ], $position: 0, $slice: 3 } } })
// segna tutte le righe con qty >= 4
db.lab_orders.updateMany({}, { $set: { 'items.$[r].flag': 'bulk' } }, { arrayFilters: [ { 'r.qty': { $gte: 4 } } ]
})
```

`upsert`, `findOneAndUpdate`, update con pipeline

- **`upsert: true`**: se nessun documento soddisfa il filtro, ne crea uno; `$setOnInsert` imposta campi solo in inserimento. Senza indice univoco sul filtro, due upsert concorrenti possono inserire **due documenti**.
- **`findOneAndUpdate`** aggiorna **e restituisce** il documento, atomicamente (`returnDocument: 'after'`): il contatore, la coda.
- **Update con pipeline**: calcola un campo a partire da altri campi dello stesso documento.

FINDONEANDUPDATE · CHIAMATE SUCCESSIVE

chiamata 1	→	`{ _id: 'order', seq: 1 }`
chiamata 2	→	`{ _id: 'order', seq: 2 }`
chiamata 3	→	`{ _id: 'order', seq: 3 }`

La prima chiamata crea il contatore (upsert), le successive lo incrementano e lo restituiscono.

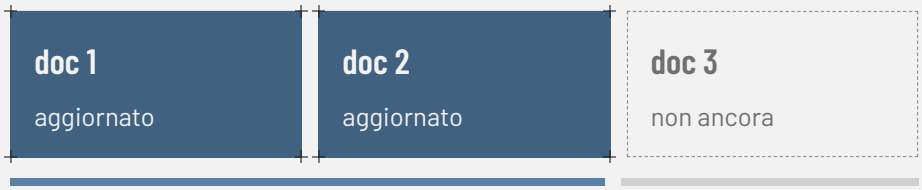
MONGOSH

```
// { _id: 'order', seq: 1 }
db.lab_counters.findOneAndUpdate({ _id: 'order' }, { $inc: { seq: 1 } }, { upsert: true, returnDocument:
'after' })
db.lab_orders.updateMany({}, [ { $set: { totalQty: { $sum: '$items.qty' } } } ])
```

`updateMany` non è atomico: update rieseguibili

- `updateMany` su N documenti produce **N operazioni**, non una. Conseguenze:
  - un lettore concorrente può vedere una parte dei documenti già aggiornata;
  - se si interrompe (errore, failover), i documenti già aggiornati **restano** aggiornati;
  - non è ritentata automaticamente dal driver.
- Quindi gli update di massa si scrivono **rieseguibili**: filtro che esclude i documenti già fatti, valori impostati con `$set` e non incrementati.
- Un array che cresce senza limite (`$push` senza `$slice`) riscrive il documento a ogni evento e arriva ai **16 MB**.

UPDATEMANY SU 3 DOCUMENTI = 3 OPERAZIONI



Un lettore a metà vede **2 documenti aggiornati e 1 no**. Se si interrompe qui, i primi due restano aggiornati.

Delete

- `deleteOne(filtro)`, `deleteMany(filtro)` (`deletedCount`). Stesse regole degli update: atomicità per documento, `deleteMany` non atomico.
- **Il filtro vuoto cancella tutto**: `deleteMany({})`. Caso reale: un filtro costruito da parametri facoltativi che, se nessuno è valorizzato, diventa `{}`.

	`deleteMany({})`	`drop()`
Documenti	eliminati uno a uno	collezione eliminata
Indici e validatore	**restano**	**eliminati**
Voci nell'oplog	**una per documento**	una sola
Tempo su collezioni grandi	proporzionale ai documenti	quasi immediato

V1 CICLO 2 · VALIDAZIONE

Schema validation: `$jsonSchema`

+ "Schemaless" è impreciso: lo schema esiste sempre, ma vive nel codice. La **validazione** lo porta nel database come **ultima difesa**, valida per **tutti** i client (applicazione, script, shell, ETL).

MONGOSH

```
db.runCommand({ collMod: 'lab_products',
  validator: { $jsonSchema: { bsonType: 'object', required: ['name', 'price'],
    properties: { name:  { bsonType: 'string', minLength: 1 },
                  price: { bsonType: 'decimal', minimum: 0 },
                  stock: { bsonType: 'int', minimum: 0 },
                  tags:  { bsonType: 'array', items: { bsonType: 'string' } } } } },
  validationLevel: 'moderate', validationAction: 'error' })
```

+ Parole chiave: `required`, `bsonType`, `enum`, `minimum`, `pattern`, `items`, `properties`, `additionalProperties`. Un documento che non rispetta lo schema è rifiutato con `Document failed validation` (codice **121**); `errInfo.details` dice quale regola e su quale campo.

+ `additionalProperties: false` rifiuta i campi non elencati: **va elencato anche `_id`**, altrimenti ogni insert fallisce.

Introdurre un validatore su dati esistenti

- `collMod` **non controlla i dati esistenti**: il validatore vale da quel momento.
- `validationLevel`:

LIVELLO	INSERT	UPDATE DI UN DOCUMENTO VALIDO	UPDATE DI UN DOCUMENTO GIÀ NON VALIDO
`strict` (default)	controllato	controllato	**controllato**: rifiutato se resta non valido
`moderate`	controllato	controllato	**non controllato**

- `validationAction`: `error` (rifiuta) oppure **`warn`** (accetta e scrive la violazione nel log, con `id` 20294): serve a misurare l'impatto prima di rendere il validatore bloccante.
- Si trovano i documenti non validi con `find({ $nor: [ schema ] })`.
- **Percorso**: `moderate` → correggere i dati storici → `strict`.
- La validazione **non** dà: integrità referenziale, univocità (serve un indice `unique`), valori di default. Il privilegio di **bypass** va dato solo agli utenti tecnici.

INTRODURRE IL VALIDATORE

1. **`warn`** · misurare l'impatto nel log
2. **`moderate`** · i nuovi dati sono controllati
3. correggere i dati storici · `$nor: [ schema ]`
4. **`strict`** · tutto controllato

V3 CICLO 2 · VALIDAZIONE

`collMod` in pratica: dal warn allo strict

+ `collMod` modifica una collezione esistente. Le opzioni sono **indipendenti**: si cambia solo `validationLevel` o `validationAction` senza riscrivere il validatore. Per rimuoverlo: `validator: {}`.

FASE 1 · INTRODURRE SENZA BLOCCARE

```
const schema = { $jsonSchema: {
  required: ['name', 'price'],
  properties: { price: { bsonType: 'decimal' }
} } }
db.runCommand({ collMod: 'lab_products',
  validator: schema,
  validationLevel: 'moderate',
  validationAction: 'warn' })
// quanti documenti storici violano lo schema?
db.lab_products.countDocuments({ $nor: [schema]
})
```

FASE 2 · CORREGGERE E RENDERE BLOCCANTE

```
// prezzi numerici → Decimal128
db.lab_products.updateMany(
  { price: { $type: ['int', 'double'] } },
  [{ $set: { price: { $toDecimal: '$price' } } }])
db.runCommand({ collMod: 'lab_products',
  validationLevel: 'strict',
  validationAction: 'error' })
// configurazione attiva
db.getCollectionInfos({ name: 'lab_products' })
[0].options
```

mongod, WiredTiger e cache

- `mongod`: **un processo**, un thread per connessione (PostgreSQL: un processo per connessione). Log in **JSON**, con `id` filtrabile.
- Storage engine **WiredTiger**: ogni collezione e ogni indice è un file `.wt` (un B-Tree); compressione **snappy** sui dati, dei prefissi sugli indici.
- **Cache**: default il maggiore tra **50% di (RAM − 1 GB)** e 256 MB (nel container, la RAM del container). In cache i dati sono **decompressi**; la cache del sistema operativo tiene i compressi.
- **Eviction**: oltre l'80% di riempimento i thread di background scaricano pagine; oltre il **95%** partecipano **anche i thread applicativi**.

UN PROCESSO, UN THREAD PER CONNESSIONE



CACHE WIREDTIGER · RIEMPIMENTO

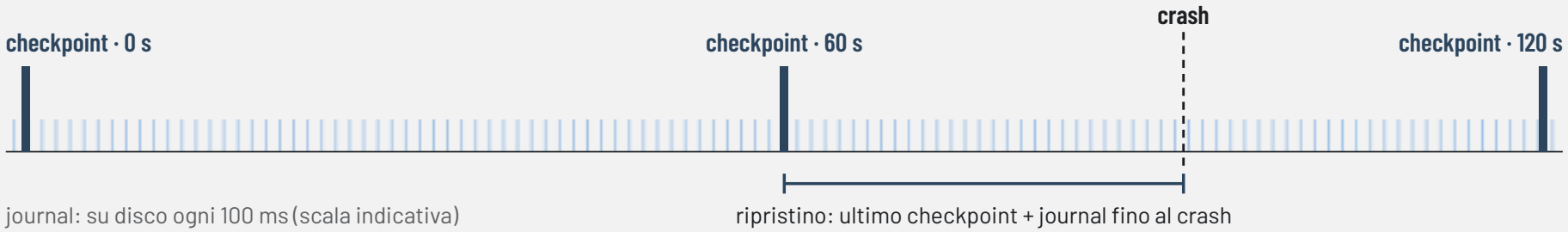


- fino all'80%: funzionamento normale
- 80–95%: eviction dai thread di background
- oltre il 95%: anche i thread applicativi → latenze

A2 CICLO 2 · ARCHITETTURA

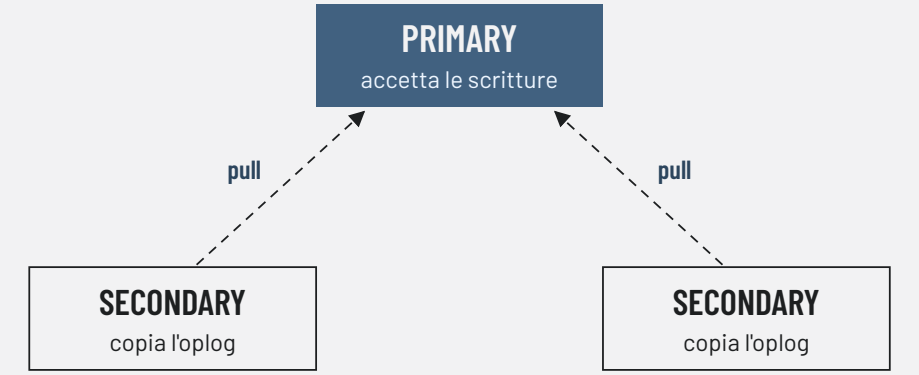
Journal, checkpoint e concorrenza: il parallelo con MVCC

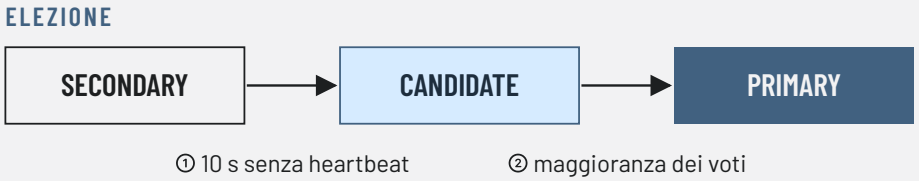
- **Checkpoint**: ogni **60 s** WiredTiger scrive un'immagine consistente dei dati. **Journal**: registro delle modifiche successive, scritto su disco ogni **100 ms** (o subito con `j: true`). Ripristino: ultimo checkpoint + journal.
- **Concorrenza a livello di documento**, ottimistica: due scritture sullo stesso documento entrano in conflitto (una viene ritentata dal server), su documenti diversi procedono in parallelo.
- Le letture lavorano su uno **snapshot**: **chi legge non blocca chi scrive**. Le versioni vecchie stanno in cache e nello **history store**, e si eliminano quando nessuno snapshot le richiede: **nessun `VACUUM`**.



Replica set: primary, secondary, elezione

- **Replica set**: `mongod` con gli **stessi dati**. Serve per alta disponibilità (failover automatico), durabilità, manutenzione a rotazione. **Non è un backup**: una cancellazione si replica.
- **Primary**: l'unico che accetta scritture. **Secondary**: copiano l'oplog del primary (modello **pull**), in modo **asincrono**. Minimo consigliato: **3 nodi con dati**.
- **Elezione**: il primary perde la maggioranza → si dimette; i secondary non ricevono heartbeat per `electionTimeoutMillis` (**10 s**) → uno si candida; vince chi ottiene la maggioranza dei voti con un oplog aggiornato. Failover tipico: **pochi secondi**. Il vecchio primary, al rientro, **annulla** (rollback) le scritture non replicate sulla maggioranza.





Il parallelo con Raft

RAFT	REPLICA SET MONGODB
leader	primary
follower	secondary
candidate	stato di candidato durante l'elezione
term	`term` (in `rs.status()` e nel campo `t` di ogni voce dell'oplog)
log replicato	oplog
voce *committed* (maggioranza)	*majority commit point*: ciò che `w: 'majority'` attende
election timeout	`electionTimeoutMillis` (**10 s**), heartbeat ogni **2 s**
voto solo a log aggiornati	vince chi ha l'oplog più aggiornato

+ Dalla 3.2 il protocollo di replica (`protocolVersion: 1`) è **basato su Raft**. Le differenze:
  - **Pull, non push**: in Raft il leader invia le voci; in MongoDB i secondary le leggono, anche da un altro secondary (*chaining*).
  - **Rollback**: le voci non confermate dalla maggioranza vengono annullate come in Raft, ma MongoDB le salva in file di rollback.
  - **Estensioni**: `priority`, membri `hidden` e `delayed`, arbiter (vota senza dati).

A4 CICLO 2 · ARCHITETTURA

L'oplog

- `local.oplog.rs`: collezione **capped** (dimensione fissa, sovrascrive le voci più vecchie). Una voce **per documento modificato**. Dimensione di default: 5% del disco libero (min **990 MB**, max 50 GB).
- Le voci sono **idempotenti**: riapplicarle una o più volte dà lo stesso risultato.
- **Finestra dell'oplog**: l'intervallo di tempo coperto. Un secondary rimasto indietro oltre la finestra rifà la **sincronizzazione completa**.
- Si legge come una collezione normale, ma non si scrive mai a mano:

MONGOSH

```
db.getSiblingDB('local').oplog.rs.find({ ns: 'shop.lab_orders' }, { op: 1, ns: 1, o: 1, _id: 0 }).sort({
$natural: -1 }).limit(1)
// dimensione e finestra dell'oplog
rs.printReplicationInfo()
```

Demo: un `$inc` nell'oplog diventa il valore finale

MONGOSH

```
db.lab_demo.insertOne({ _id: 'A', qty: 10 })
db.lab_demo.updateOne({ _id: 'A' }, { $inc: { qty: 1 } })
db.getSiblingDB('local').oplog.rs.find({ ns: 'shop.lab_demo', op: 'u' }, { o: 1, o2: 1, _id: 0 }).sort({ $natural: -1 }).limit(1)
```

OUTPUT ATTESO

```
[ { o: { '$v': 2, diff: { u: { qty: 11 } } }, o2: { _id: 'A' } } ]
```

+ L'oplog contiene il **valore risultante** (`qty: 11`), non l'incremento (`+1`).

COMANDO → VOCE DELL'OPLOG

`{ $inc: { qty: 1 } }` — comando

↓ registrato come valore risultante

`{ u: { qty: 11 } }` — voce

☐ riapplicata due volte → sempre **11**

■ se fosse "+1": **12** al secondo replay

A6 CICLO 2 · ARCHITETTURA

Write concern e read concern

WRITE CONCERN	SIGNIFICATO
`w: 1`	confermata dal primary
`w: 'majority'`	confermata dalla **maggioranza** (**default dalla 5.0**)
`w: <n>`	confermata da n membri
`j: true`	scritta nel journal prima della conferma
`wtimeout`	tempo massimo di attesa: **non annulla** la scrittura

MONGOSH

```
db.orders.insertOne(
  { _id: 900001, status: 'paid', total: NumberDecimal('129.90') },
  // attende maggioranza + journal, max 5 s
  { writeConcern: { w: 'majority', j: true, wtimeout: 5000 } }
)
// solo dati non annullabili
db.orders.find({ status: 'paid' }).readConcern('majority')
// nodo singolo: errore 100, ma l'update resta
db.orders.updateOne({ _id: 900001 }, { $set: { status: 'shipped' } },
  { writeConcern: { w: 2 } })
```

READ CONCERN	RESTITUISCE
`local` (**default**)	i dati più recenti del nodo, anche se non ancora replicati sulla maggioranza
`majority`	solo dati confermati dalla maggioranza: **non verranno annullati** da un rollback
`linearizable`	dati confermati e successivi a ogni scrittura confermata (solo sul primary, costoso)
`snapshot`	tutte le letture sullo stesso istante (transazioni)

Con un nodo solo, `w: 2` **non si può soddisfare**: `MongoWriteConcernError[UnsatisfiableWriteConcern]` (codice 100). La scrittura **resta**.

A7 CICLO 2 · ARCHITETTURA

Read preference, configurazioni tipo e ritorno a CAP

+ **Read preference**: `primary` (default), `primaryPreferred`, `secondary`, `secondaryPreferred`, `nearest`. Leggere dai secondary **non aumenta la capacità di scrittura** e può restituire dati non aggiornati.
+ Si sceglie **per tipo di dato**:

CASO	WRITE CONCERN	READ CONCERN	READ PREFERENCE
Dati critici (ordini, pagamenti)	`majority`	`majority`	`primary`
Letture veloci, ritardo tollerato (catalogo)	`majority`	`local`	`secondaryPreferred`
Dati sacrificabili ad alto volume (telemetria)	`w: 1`	`local`	`primary`

+ **CAP/PACELC**: con `majority` + `primary` il lato senza maggioranza **non accetta scritture** (PC/EC); con `w: 1` e letture dai secondary la risposta è rapida ma può essere non aggiornata (EL).

04

CICLO 2 · LABORATORIO

Modifica, validazione, replica

Otto esercizi su ShopIT: update, validazione dello schema, replica. In chiusura, correzione in plenaria.

L2 CICLO 2 · CONSEGNA

Consegna Laboratorio 2

+ **Fase 2: modifica, validazione, replica** sul database `shop` di ShopIT.

TEMA	#	ESERCIZIO
Update	1	Vendi 3 pezzi di `P-001` **solo se** lo stock basta, in un'unica operazione. Ripeti finché `matchedCount` è 0.
Update	2	Aggiungi il tag `'promo'` a `P-002` due volte: con `$push` e con `$addToSet`. Quanti tag restano?
Update	3	Tieni in `P-003` solo le **ultime 3** recensioni, la più recente in testa. Poi segna con `flag: 'bulk'` le righe con `qty` ≥ 4 di un ordine.
Update	4	Contatore `order` con `findOneAndUpdate` e `upsert`: chiamalo tre volte. Poi scrivi un `updateMany` rieseguibile che imposta `totalQty`.
Validazione	5	Validatore `$jsonSchema` su `lab_products` (`name`, `price` decimal ≥ 0). Inserisci `price: 10`: leggi codice e `errInfo`.
Validazione	6	Trova i prodotti non validi con `$nor`, prova un update su `P-006` in `moderate` e in `strict`, correggi i dati, passa a `strict`.
Replica	7	Un `$inc` sulla giacenza di un prodotto: trova la voce nell'oplog e spiega perché contiene il valore finale.
Replica	8	Inserisci con `w: 2` sul nodo singolo: che errore ricevi? Il documento c'è?

R1 RECAP

Recap della mattina

TEMA	DA RICORDARE
Paradigma	il documento è l'aggregato, atomico per definizione; CAP/PACELC si scelgono per operazione
BSON	`int`/`double` dipendono dal valore; `Decimal128` per gli importi; `Date`, non stringhe
Insert e Find	`ordered: false` per i caricamenti; i negativi includono l'assente; `$elemMatch` per la stessa riga; paginazione per range
Update	la condizione nel filtro rende l'update sicuro; `updateMany` non è atomico: update rieseguibili
Validazione	`$jsonSchema` come ultima difesa; `moderate` → correggere → `strict`
Architettura	oplog idempotente; write/read concern per operazione; WiredTiger senza `VACUUM`